

Course Planning & Advising System

"ระบบวางแผนการเรียนและช่วยเหลืออาจารย์ที่ปรึกษา มุ่งเน้นการตรวจสอบเงื่อนไขวิชาบังคับก่อน (Prerequisites) และการจัดการตารางเรียนสำหรับวิชาที่สอบตก ด้วยสถาปัตยกรรมที่ครอบคลุมเขตได้ 100% โดยไม่พึ่งพาระบบลงทะเบียนภายนอก"

1. ภาพรวมระบบและสถาปัตยกรรม (System Architecture)

ระบบนี้ถูกออกแบบขึ้นเพื่ออำนวยความสะดวกให้นักศึกษาในการวางแผนการเรียน โดยเฉพาะกลุ่มนักศึกษาที่ต้องการลงทะเบียนเรียนแก้ตัวในรายวิชาที่สอบตก (Failed Courses) รวมถึงช่วยให้อาจารย์ที่ปรึกษาสามารถเข้าถึง ตรวจสอบ และปรับปรุงแผนการเรียนของนักศึกษาในความดูแลได้อย่างมีประสิทธิภาพ เพื่อหลีกเลี่ยงปัญหาด้านข้อจำกัดทางเทคนิคและความปลอดภัยของข้อมูล (PDPA) ระบบจะทำงานแบบ **Self-contained** โดยไม่ต้องเชื่อมต่อ API โดยตรงกับระบบสารสนเทศของมหาวิทยาลัย (SIS) แต่จะใช้การอัปโหลดไฟล์ใบแสดงผลการเรียน (Transcript) และให้อาจารย์ที่ปรึกษาเป็นผู้จัดการข้อมูลรอบการเปิดสอนของวิชาต่าง ๆ แทน

เทคโนโลยีที่ใช้ (Tech Stack)

- Backend Framework:** Nest.js (Node.js) ร่วมกับ TypeScript
- Database:** PostgreSQL (เชื่อมต่อผ่าน TypeORM หรือ Prisma)
- Authentication:** Passport.js ร่วมกับ JWT (JSON Web Tokens)
- Parsing Engine:** pdf-parse (สำหรับดึงข้อความจากไฟล์ Transcript PDF)

2. ระบบจัดการผู้ใช้งานและสิทธิ์การเข้าถึง (User Roles & Auth)

ระบบแบ่งผู้ใช้งานออกเป็น 2 บทบาทหลัก (Roles) โดยใช้สิทธิ์ในการควบคุมการเข้าถึงข้อมูลด้วยระดับ Role-Based Access Control (RBAC) ซึ่ง Nest.js จะใช้ Custom Guards (เช่น `@UseGuards(JwtAuthGuard, RolesGuard)`) ในการป้องกัน Endpoint

บทบาท (Role)	สิทธิ์การเข้าถึงและการกระทำ (Permissions)	ข้อจำกัดความปลอดภัย (Security Constraints)
Student (นักศึกษา)	- อัปโหลด Transcript ของตัวเองเพื่อนำข้อมูลเข้าระบบ - จัดวางแผนการเรียนของตนเอง - ดูคำแนะนำวิชาที่ควรลงทะเบียนเรียนแก้ตัว	ไม่สามารถ เข้าถึงแผนการเรียนหรือข้อมูลของนักศึกษาคนอื่นได้ - เข้าถึงได้เฉพาะ endpoints ที่ระบุสิทธิ์ <code>'student'</code>
Advisor (อาจารย์ที่ปรึกษา)	- ค้นหาและดูรายชื่อนักศึกษาในความดูแล (Advisees) - ดูและแก้ไขแผนการเรียนของนักศึกษาในความดูแลได้โดยตรง - จัดการข้อมูลวิชาเรียน เช่น กำหนดปี/เทอมที่วิชาเปิดสอน (Admin UI)	ไม่สามารถ แก้ไขแผนการเรียนของนักศึกษาที่ไม่ได้อยู่ในความดูแลได้ (ต้องใช้ Guard ระดับ Row-Level เพื่อตรวจสอบความสัมพันธ์)

ตัวอย่างการป้องกัน Endpoint บน Nest.js Controller:

```
@Post('student/:id/plan')
@Roles('student', 'advisor')
@UseGuards(JwtAuthGuard, RolesGuard, OwnerOrAdvisorGuard)
async updatePlan(@Param('id') studentId: string, @Body() planData: UpdatePlanDto) {
    return this.planService.updateStudentPlan(studentId, planData);
}
```

3. ระบบดึงข้อมูลเกรดอัตโนมัติ (Transcript Parser Engine)

เนื่องจากข้อจำกัดด้านความปลอดภัยของข้อมูลส่วนบุคคล (PDPA) และความยากในการขอสิทธิ์เข้าถึง API ของมหาวิทยาลัย ระบบจึงเปลี่ยนมาใช้แนวทาง **"Student-Driven Import"** โดยให้นักศึกษาดาวน์โหลด Transcript (PDF หรือ CSV) จากระบบของมหาวิทยาลัย (เช่น ระบบ SIS ของมหาวิทยาลัยแม่โจ้) แล้วนำมาอัปโหลดเข้าระบบเพื่อวิเคราะห์รายวิชาที่ตก (ได้เกรด F, U, หรือ W)

ขั้นตอนการทำงานของ Parser (Workflow)

1. **Upload Action:** นักศึกษาอัปโหลดไฟล์ `transcript.pdf` ผ่าน UI
2. **Text Extraction:** Backend (Nest.js) ใช้ไลบรารี `pdf-parse` ในการแปลงข้อมูล PDF ออกมาเป็น Raw Text
3. **Regex Pattern Matching:** ใช้ Regular Expression ในการจับกลุ่มคำ (Pattern) เพื่อดึงข้อมูล:
 - รหัสวิชา (Course Code) เช่น `ศน101` , `MJU101` , `CS311`
 - ชื่อวิชา (Course Title)
 - จำนวนหน่วยกิต (Credits)
 - เกรดที่ได้ (Grade) โดยเน้นสแกนหาตัวอักษร `F` , `U` , `W`
4. **Database Sync:** นำวิชาที่สอบตกเข้าระบบเพื่อบันทึกเป็นสถานะ `failed_courses` และใช้เป็นข้อมูลตั้งต้นสำหรับการวางแผนการเรียนต่อไป

แนวทางการเขียน Regex สำหรับวิเคราะห์ Transcript (ตัวอย่างมหาวิทยาลัยแม่โจ้)

```
// ตัวอย่างรูปแบบข้อความใน Transcript: "CS 311 SYSTEM ANALYSIS AND DESIGN 3.0 F"
const courseRegex = /([A-Z]{2,3}\s*\d{3})\s+(.+?)\s+(\d\.\d)\s+([A-F,U,W]{1,2})/g;

// ตัวอย่างผลลัพธ์การจับคู่:
// Group 1 (รหัสวิชา): "CS 311"
// Group 2 (ชื่อวิชา): "SYSTEM ANALYSIS AND DESIGN"
// Group 3 (หน่วยกิต): "3.0"
// Group 4 (เกรดที่ได้): "F" (ระบบจะทำเครื่องหมายวิชานี้เพื่อร่อนำเสนอแผนแก้ตัว)
```

4. ระบบคำแนะนำช่วงเวลาเรียนแก้ตัว (Failed Course Scheduler Engine)

การแนะนำตารางลงทะเบียนสำหรับรายวิชาที่สอบตกจะไม่ใช้การอิงข้อมูลแบบ Dynamic จากมหาวิทยาลัย แต่จะอ้างอิงจากฟิลด์ข้อมูลที่เพิ่มขึ้นมาใหม่ใน Entity ของวิชาเรียนเพื่อให้ระบบสามารถจัดสรรแบบคงที่ (Static) ได้อย่างแม่นยำ

การออกแบบโครงสร้างฟิลด์ระบุรอบเทอมการเปิดสอน (Offered Semesters Mapping)

ในโมเดล `Course` จะเพิ่มฟิลด์ `offeredSemesters` ซึ่งเก็บข้อมูลเป็น Array ของตัวเลขเทอมที่เปิดสอนจริง เพื่อรองรับความหลากหลายของธรรมชาติรายวิชา (เช่น บางวิชาเปิดเฉพาะเทอม 1 บางวิชาเปิดทุกเทอม) ดังตารางด้านล่าง:

รหัสวิชาตัวอย่าง	ชื่อวิชา	offeredSemesters	ความหมายรอบการเปิดสอน
CS 100	Computer Computer and Information	[1, 2]	เปิดสอนทั้ง เทอม 1 และ เทอม 2
CS 311	System Analysis and Design	[1]	เปิดสอนเฉพาะ เทอม 1 เท่านั้น
CS 497	Cooperative Education	[1, 2, 3]	เปิดสอนทุกเทอมรวมถึง เทอมฤดูร้อน (3)

ขั้นตอนการคำนวณแผนการลงทะเบียนเรียนใหม่ (Recommendation Logic)

เมื่อพบรายวิชาที่ติดสถานะ **F** หรือสอบตก ระบบจะใช้อัลกอริทึมในการแนะนำช่วงเวลากลับมาลงทะเบียนเรียนใหม่ ดังนี้:

- ตรวจสอบ Prerequisites:** ค้นหาโครงสร้างต้นไม้ข้อกำหนดวิชาเรียน (Prerequisite Tree) ว่าวิชาที่ตกนี้ เป็นวิชาบังคับก่อนของวิชาอื่นในแผนการเรียนรอบถัดไปหรือไม่ (ถ้าเป็นวิชาบังคับก่อนระดับวิกฤต จะต้องถูกจัดตารางให้เรียนแก้ตัวกันทีในเทอมที่เปิดสอนถัดไปที่เร็วที่สุด)
- สอดคล้องกับรอบที่เปิดจริง:** ค้นหาเทอมถัดไปในรอบปีการศึกษาของนักศึกษาที่ตรงกับค่าใน **offeredSemesters**
- การคูณภาระหน่วยกิตสูงสุด:** แนะนำวิชาแก้ตัวโดยคำนวณไม่ให้หน่วยกิตรวมในเทอมนั้น ๆ เกินโควตาที่ระบุไว้ (เช่น ไม่เกิน 22 หน่วยกิตในเทอมปกติ หรือ 9 หน่วยกิตในเทอมฤดูร้อน)

5. โครงสร้างฐานข้อมูล (Database Schema Specifications)

ด้านล่างนี้คือโครงสร้างความสัมพันธ์ (Entity-Relationship Design) ในรูปแบบ TypeScript TypeORM เพื่ออธิบายสคีมาของระบบทั้งหมด

5.1 User Entity (ผู้ใช้งานระบบ)

```
@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  username: string; // รหัสนักศึกษา หรือ ชื่อลืออีกนอาจารย์

  @Column()
  passwordHash: string;

  @Column({ type: 'enum', enum: ['student', 'advisor'] })
  role: 'student' | 'advisor';

  @Column()
  fullName: string;

  // สำหรับนักศึกษา: ระบุดีของอาจารย์ที่ปรึกษา
  @ManyToOne(() => User, (user) => user.advisees, { nullable: true })
  advisor: User;

  // สำหรับอาจารย์ที่ปรึกษา: ลิสต์นักศึกษาในความดูแล
  @OneToMany(() => User, (user) => user.advisor)
  advisees: User[];
}
```

5.2 Course Entity (วิชาเรียนและรอบเวลาเปิดสอน)

```
@Entity('courses')
export class Course {
  @PrimaryColumn()
  code: string; // รหัสวิชา เช่น "CS311"

  @Column()
  title: string;

  @Column({ type: 'int' })
  credits: number;

  @Column({ type: 'simple-array' })
  offeredSemesters: number[]; // เช่น [1, 2] หมายถึง เปิดสอน 1 และ เทอม 2

  // ความสัมพันธ์ Prerequisite (วิชาที่ต้องเรียนก่อน)
  @ManyToMany(() => Course)
  @JoinTable({ name: 'course_prerequisites' })
  prerequisites: Course[];
}
```

5.3 StudyPlan & FailedCourse Entities (แผนการเรียนและประวัติวิชาที่สอบตก)

```
@Entity('study_plans')
export class StudyPlan {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @ManyToOne(() => User)
  student: User;

  @Column({ type: 'int' })
  academicYear: number; // เช่น 2569

  @Column({ type: 'int' })
  semester: number; // 1, 2 หรือ 3 (ฤดูร้อน)

  @ManyToMany(() => Course)
  @JoinTable({ name: 'study_plan_courses' })
  selectedCourses: Course[];
}

@Entity('failed_courses')
export class FailedCourse {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @ManyToOne(() => User)
  student: User;

  @ManyToOne(() => Course)
  course: Course;

  @Column({ default: false })
  isResolved: boolean; // มีสถานะเป็น true เมื่อนักศึกษาลงทะเบียนเรียนใหม่และสอบผ่านแล้ว
}
```

6. ข้อมูลทางเทคนิคและการส่งต่อข้อมูล (API Specifications)

นี่คือข้อมูลโครงสร้างการแลกเปลี่ยนข้อมูล (Request & Response Payload) สำหรับการนำไปพัฒนาในส่วนของการเชื่อมต่อระบบ ฝั่ง Frontend และ Backend

6.1 อัปโหลดไฟล์ Transcript (PDF) เพื่อประมวลผลวิชาที่ตก

POST /api/transcript/upload

Headers: Content-Type: multipart/form-data , Authorization: Bearer <JWT>

Request Body:

```
{
  "file": [Binary PDF File]
}
```

Response Payload (201 Created):

```
{
  "success": true,
  "studentId": "6504101301",
  "parsedFailedCoursesCount": 2,
  "failedCourses": [
    {
      "code": "CS 311",
      "title": "SYSTEM ANALYSIS AND DESIGN",
      "credits": 3,
      "grade": "F",
      "offeredSemesters": [1]
    },
    {
      "code": "MTH 102",
      "title": "CALCULUS II",
      "credits": 3,
      "grade": "F",
      "offeredSemesters": [2]
    }
  ]
}
```

6.2 บันทึกข้อมูลและจัดแผนการเรียนใหม่โดยอาจารย์หรือนักศึกษา

PUT /api/students/:studentId/plans

Headers: Content-Type: application/json , Authorization: Bearer <JWT>

Request Body:


```
{
  "academicYear": 2569,
  "semester": 1,
  "courseCodes": ["CS 311", "CS 312", "En 102"]
}
```

Response Payload (200 OK):

```
{
  "success": true,
  "message": "Study plan successfully updated",
  "plan": {
    "id": "e4a8b79d-3f12-4c20-a612-411a0b387e33",
    "studentId": "6504101301",
    "academicYear": 2569,
    "semester": 1,
    "courses": [
      { "code": "CS 311", "title": "SYSTEM ANALYSIS AND DESIGN", "credits": 3 },
      { "code": "CS 312", "title": "SOFTWARE ENGINEERING", "credits": 3 },
      { "code": "En 102", "title": "ENGLISH FOR COMMUNICATION", "credits": 3 }
    ],
    "totalCredits": 9
  }
}
```